

Action Primitive Library for the GoBot

Lauren Gayle

Introduction

A challenge of early motor intervention is motivating children to stay engaged and active during their sessions. Robots offer a means to boost engagement and encourage movement of children. Assistive robots for child development has been seen in many works before such as Fitter et al.'s socially assistive robot to encourage leg-motion training in infants [3], Marino et al.'s use of a robot for teaching emotional understanding [2], and Bugnariu et al.'s use of a robot to quantify motor imitation behavior in children with autism spectrum disorder [1]. The assistive robot GoBot is an assistive robot meant to encourage movement in children through play, first proposed in [4]. The GoBot has the ability to play "keep away" and "chase" with the child, staying away from or moving towards a child, as well as encourage the child with the use of a rewards stack module containing lights, sounds, and possible bubbles or tossing a ball. All of these possible actions can be executed independently or in tandem, and the execution of actions is handled using a behavior tree.

The GoBot has already proved itself effective in many regards, however an increase in the GoBot's capabilities to move and play would ultimately boost the effectiveness of the GoBot's ability to encourage motion in children. In order to boost the manner in which the GoBot can be encouraging, a library of action primitives was introduced to the GoBot. These action primitives consist of clockwise and counterclockwise circles, forwards and backwards linear motion, and clockwise and counterclockwise rotation as depicted in Figure 1. These action

primitives can be combined with one another into several possible paths for the GoBot to execute, and this motion can be utilized to encourage motion of the child.

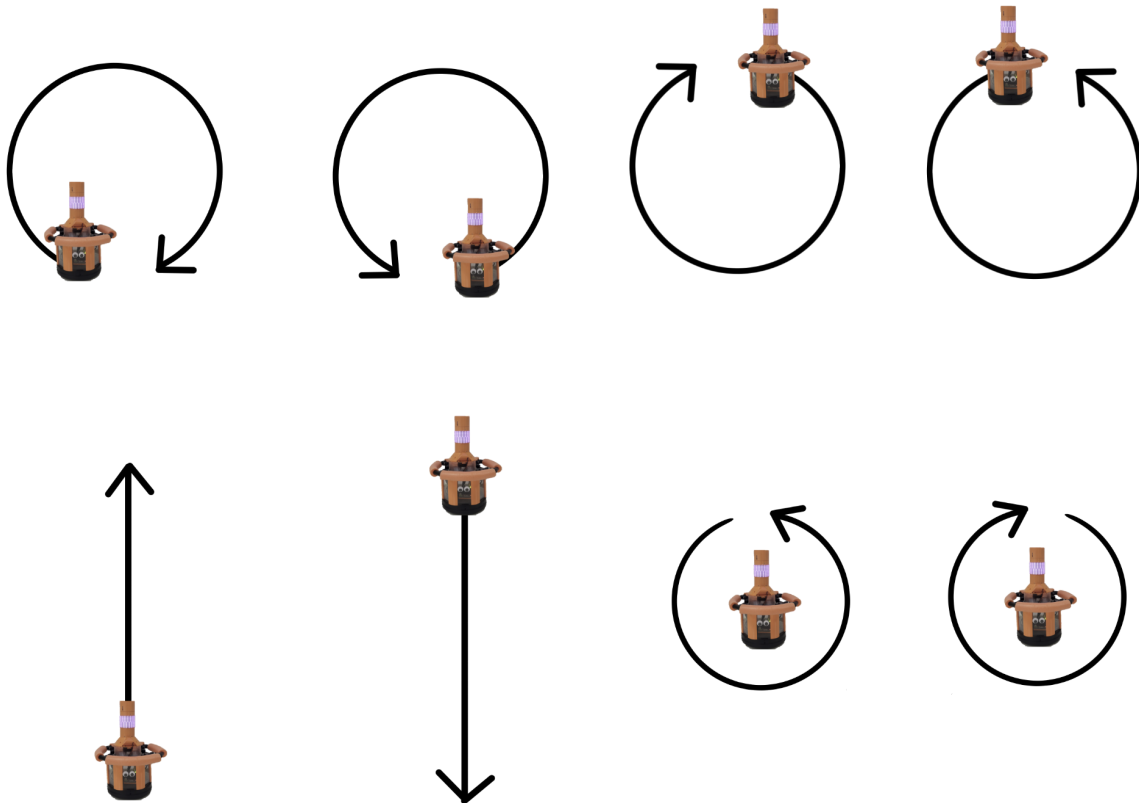


Figure 1: Action primitive library

This paper lays out how the action primitive library is implemented into the GoBot's code, focusing on how it was made and how it can be utilized to make more complex motions. As well as points to future possible steps for the action primitive library.

Library Implementation

To start, the files containing the twist commands that dictate the motion of the GoBot when the primitive is called are found in `lidar_e/scripts/action_primitives/` in the GoBot code

found [here](#). There are eight files, each one with a name corresponding to the motion they deal with, as well as a README explaining the action primitive library, how to make one, and how to combine them into a path.

The motion files themselves use the publisher variable “pub” to publish the twist commands, the publisher variable “close_pub” to publish the closest object reading from the LiDAR sensor, both of which are created at the top of the file, and the boolean subscriber, which shares a name with the file and is instantiated in the listener function, which tells if the action is active or not. The most important information occurs in the main callback block in the cc function. Within this function, the object that is closest based on the LiDAR reading is determined, and, if the game is active, twist commands are published based on the closest object's distance. If the object is greater than half a meter away from the sensor, the GoBot will go about it's commanded action, and if an object is less than half a meter away, the GoBot will reverse the path it is on if it is a linear or circular path, meaning if it is moving forwards it will go backwards, and if it is a rotation motion the GoBot will just stop moving until the close object is at least half a meter away again.

In order to use these action primitive files on the GoBot, they must be called in the games.launch file in lidar_e/launch/ in the GoBot's code. This listens to the subscriber made in the listener function in the action primitive file, making it accessible to the rest of the files. The action primitive must also be instantiated within turtlebot_controller.py, found in turtlebot_controller/src/. This will ensure that when the twist commands are called for the action primitives they do not conflict with the controller for the GoBot. This is done by making a global boolean variable to determine if the action is on or not in the turtlebot_controller function, setting that boolean to false, and then in a conditional statement at the bottom of the function, the

boolean is placed in the conditions to ensure that when the action is false the controller can send commands. The action primitives can also be tied to buttons in this file, though that is not a necessity for using them to create more robust motion, as that all happens within a robot.py file.

Once the file is made accessible by `games.launch` and placed in `turtlebot_controlller.py` to ensure no conflicts with the controller, the primitives are instantiated in one of the robot.py files in `behavior_tree_autonomy/src/behavior_tree_autonomy`. The robot.py file of choice just depends on the one chosen to run on the GoBot, and the primitives are currently instantiated in `robot_testing.py`. For the primitives themselves, a boolean must be made in the `__init__` function of the Robot class that will be used to turn off and on the action, and a publisher is needed for each action to publish if the action is on or off. Then a function must be made that when called activates or deactivates the action. From there the action can be made into a singular action for the GoBot to run, or combined with one another into a path.

In order to create an action that can be placed in a behavior tree and then run by the GoBot, a constant must first be made at the top of the Robot class in a robot.py file. Then, after the act function in the file, a function can be made for the action one wishes to instantiate. An example is the `L_shape` function found on line 994 of `robot_testing.py`. Within this function the `straight_up` and `turn` action primitives are combined together. The switching between these primitives is based on duration, a variable that is used in every action on the GoBot to dictate how long an action is. In the `L_shape` function, the robot activates the `straight_up` action for a duration of 5, then turns it off and activates the `turn` action for a duration of 5 before the game ends, and resets. If it is called twice it will make an L shape, and if it is called four times it will make a square. This function is a simple example of the combination of primitives, but the

functions in `robot_testing.py` such as `figure_8`, `squiggle`, and `zig_zag` demonstrate more complex combinations of the primitives. All of the paths made as an example are seen in Figure 2.

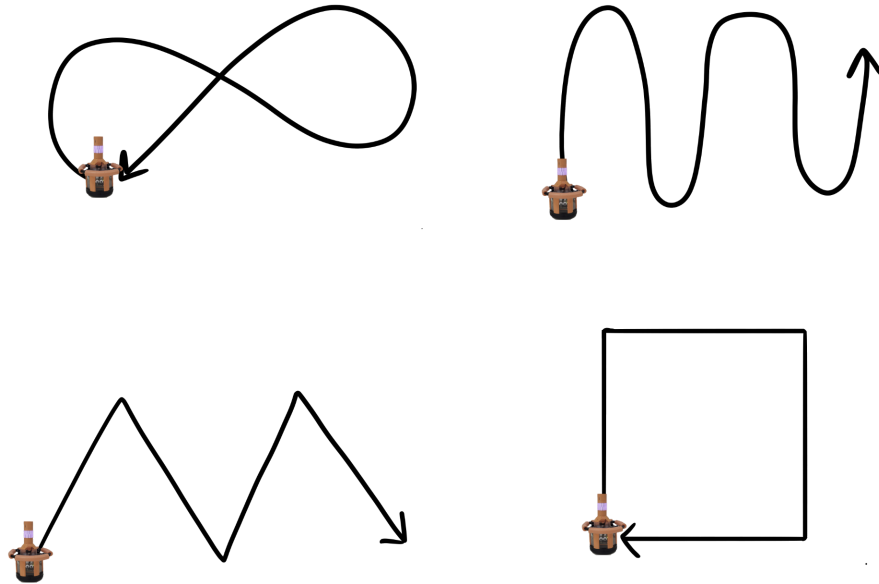


Figure 2: Example Paths

This new path is then placed in the `act` function in a conditional that checks if the `current_action` variable is equal to the constant for the action, and if so it will call the action function, and also in the `get_current_action` function, which checks if the action's name is in active actions and that the action node status for the action is equal to 1. If both of these conditions return true, the `current_action` variable is set to the constant for the called action and the `current_action_label` is set to the name of the action. Overall, a primitive is called for a set duration before switching to another and this is how they are combined to make paths. They are placed in the conditional for the `act` function and the `get_current_action` function, and this is how the path's functions are called. They are then placed in a behavior tree in `behavior_tree/config/` and that behavior tree is

called at the bottom of a robot.py file in the variable “bt_path”, and that is how they are run on the GoBot.

Functionality

These action primitives work in that they have been proven to successfully create complex paths. The paths that they have currently been used to make are a square, figure eight, squiggle line, and zig-zag line, and they are fully capable of being made into various other actions. However, issues arise when it comes to their usage of LiDAR to detect a close object. The LiDAR on the GoBot is extremely noisy, and will have incorrect readings on occasion, causing it to not backtrack its path as it should when an object is too close, or incorrectly detect an object as being close when one is not.

Future Work

There are two things that can be done to further the action primitive library. One is to improve its usage of sensors by incorporating the overhead camera it utilizes, and another is to test the effectiveness of these motion paths in encouraging the motion of children.

In terms of sensor improvement, the GoBot currently only uses the LiDAR for the action primitive library. It uses the LiDAR based on how the old keep away and chase actions are formatted to do so, however both of those now use both the overhead camera and the LiDAR. The action primitives could be edited to function like either of those games do. This would allow for more precision when it comes to gauging the distance a child is from the GoBot. The LiDAR also can detect distance, but half a meter from the front of the GoBot reads the same as half a meter behind to GoBot in the action primitive files, and this could be dealt with using the overhead camera as that is what is done in the keep away that uses the overhead camera. The

LiDAR is not so noisy that it is unsafe to run a path made by the action primitives, and the paths move so slow that they pose no real problem if they misdetect something, but it does affect path consistency.

In addition to improving the sensing, the action primitive library could be tested to see how effective more motion for the GoBot is in encouraging motion in children. This would look like running a motor intervention session as typical, but running the GoBot with a behavior tree that includes various paths, and seeing the score these paths gain for encouraging movement. These paths could also be combined with some of the reward stack functions, and run on a behavior tree to see how that encourages motion as well. Ultimately, the next step for the action primitive library is to test how effective the paths it creates are.

Conclusion

All in all, the GoBot is an assistive robot meant to encourage motion during motor interventions for children via play. The more mobility the GoBot has, the greater chance it has at encouraging movement in children. To address this, a library of action primitives was implemented into the GoBot's code that can be used to create various paths and, in turn, increase the GoBots mobility. There are many steps to implementing an action primitive itself, and then they can be combined into paths based on duration in the robot.py file executed by the GoBot. While this library has been successfully utilized to make paths, the LiDAR sensor the GoBot currently uses for them is noisy and can be inaccurate sometimes as a result. For future steps for the action primitive library, it can be made to incorporate the overhead camera so that distance can be gauged better, as well as various motion paths can be run to see how effective these paths are in encouraging motion.

Bibliography

- [1] N. Bugnariu et al., “Human-robot interaction as a tool to evaluate and quantify motor imitation behavior in children with Autism Spectrum Disorders,” in 2013 International Conference on Virtual Rehabilitation (ICVR), Aug. 2013, pp. 57–62. doi: 10.1109/ICVR.2013.6662088.
- [2] F. Marino et al., “Outcomes of a Robot-Assisted Social-Emotional Understanding Intervention for Young Children with Autism Spectrum Disorders,” *J Autism Dev Disord*, vol. 50, no. 6, pp. 1973–1987, Jun. 2020, doi: 10.1007/s10803-019-03953-x.
- [3] N. T. Fitter et al., “Socially Assistive Infant-Robot Interaction: Using Robots to Encourage Infant Leg-Motion Training,” *IEEE Robotics & Automation Magazine*, vol. 26, no. 2, pp. 12–23, Jun. 2019, doi: 10.1109/MRA.2019.2905644.
- [4] Ameer Helmi, Emily Scheide, Tze-Hsuan Wang, Samuel W. Logan, Geoffrey A. Hollinger, and Naomi T. Fitter. Gobot An autonomous assistive robot using behavior trees to encourage child mobility. Association for Computing Machinery, 2023.